

Avances del Proyecto

Avances del Proyecto — Sistema de IA Anticipatoria para el STC Metro

Alumno: Alan Bellon García **Asesor:** M. en Fil. C. Enrique Francisco Soto Astorga **Fecha de este reporte:** 2026-09-27 (actualizado — ver [Sección 4](#) y [Sección 5](#))

Este documento resume el estado técnico y metodológico del prototipo descrito en el anexo de titulación ("Modelado y prototipado de un sistema de Inteligencia Artificial Anticipatoria para la estimación de estados de congestión en el STC Metro de la Ciudad de México"), con base en el código y los datos actualmente presentes en `00_Programas/tesis_metro_ai/`.

1. Mapeo contra los objetivos de la tesis

Objetivo secundario (Anexo)	Estado	Evidencia en código
Generar dataset sintético de afluencia/disrupciones	✅ Completo (14 días sintéticos, laboral+fin de semana, eventos estocásticos)	<code>generador_sintetico_horario.py</code> , <code>simulador_congestion.py</code> , <code>datos_procesados/*.csv</code>
Representar la red como grafo con pesos	✅ Completo	<code>grafo_metro.py</code> → <code>grafo_base_metro.gexf</code>
Modelo de estimación a horizonte corto (10–60 min)	⚠️ Parcial (horizonte discreto de 1 hora, no continuo 10-60 min); modelo elegido por comparación (GradientBoosting + línea/tramo)	<code>entrenador_anticipatorio.py</code> → <code>modelos/modelo_anticipatorio.pkl</code>
Algoritmo de ruteo que integre la métrica predictiva	✅ Prueba de concepto funcional	<code>ruteo_anticipatorio.py</code>
Integración estimación + ruteo en prototipo funcional	✅ Evaluado sistemáticamente: 27 horas con evento en días no vistos, 84,650 casos O-D; la IA captura 64.6% del ahorro máximo posible (Sección 5.3)	<code>ruteo_anticipatorio.py</code>
Explicabilidad de las recomendaciones	✅ Iniciado (importancia por permutación, agnóstica al modelo)	<code>entrenador_anticipatorio.py</code> → <code>importancia_variables.png</code>
Sistema reactivo de comparación (índice 5.5)	✅ Implementado y comparado: estático vs. reactivo vs. anticipatorio vs. oráculo	<code>ruteo_anticipatorio.py</code>

En términos del índice tentativo, el proyecto ha cubierto el **capítulo 4** completo (modelado y generación de datos) y tiene un **primer corte funcional del capítulo 5** (diseño del prototipo: arquitectura, modelo predictivo, ruteo, baseline reactivo). Falta evaluación formal de KPIs (5.4), profundizar explicabilidad (6.5) y ejecutar comparativas sistemáticas (6.4) sobre múltiples escenarios, no solo el caso de prueba único.

2. Pipeline técnico actual (orden de ejecución)



2.1 grafo_metro.py — Modelado topológico

- Filtra GTFS masivo del Valle de México por `agency_id == "METRO"` para aislar rutas, viajes y `stop_times`.
- Construye grafo no dirigido $G(V, E, W)$ con **195 nodos** (andenes/estaciones).
- Peso estático de cada arista = promedio de `departure_sec` → `arrival_sec` entre paradas consecutivas del mismo `trip_id`, filtrando deltas ilógicos (< 0 o $> 1800s$).
- Agrega aristas de **transbordo peatonal** (5 min fijos) entre andenes que comparten `stop_name`, habilitando cambios de línea en el grafo.
- Exporta a `grafo_base_metro.gexf` (formato estándar, reutilizable por NetworkX).

Limitación notable: el penalti de transbordo es una constante (5 min) igual para todas las estaciones de correspondencia, sin distinguir estaciones con transbordos largos conocidos (p. ej. Ciudad Azteca, La Raza, Tacubaya).

2.2 `generador_sintetico_horario.py` — Motor de demanda

- Limpia encoding (latin1 → utf-8) de estación/línea provenientes del portal de datos abiertos.
- Clasifica estaciones en 3 perfiles espaciotemporales (`origen` , `destino` , `mixto`) mediante listas fijas (hardcoded) de 10 estaciones por extremo.
- Desagrega afluencia diaria a horaria multiplicando por curvas bimodales (`peso_origen` , `peso_destino` , `peso_mixto`) calibradas manualmente para 24 horas.
- **Matriz Origen-Destino sintética** vía modelo gravitacional simplificado: calcula `atractividad_destino` según hora y perfil, normaliza por estación de origen y reparte las entradas reales observadas como viajes probables a cada destino.
- **[Actualizado 2026-09-27]** Parametrizado por `DIAS_SIMULACION` (lista de fechas, hoy 14 días de enero 2026). Cada fecha se etiqueta `laboral` / `fin_de_semana` según su día de la semana; el fin de semana usa un perfil horario derivado programáticamente del laboral (aplanado de picos + desplazamiento de actividad a mediodía/tarde), no una segunda tabla de constantes inventada. Genera una `matriz_od_sintetica_<fecha>.csv` por día y un `manifiesto_dias_simulados.csv` (fecha, tipo_dia, archivo) que `simulador_congestion.py` consume para no duplicar la lista de días entre scripts.

2.3 `simulador_congestion.py` — Motor de estrés dinámico

- Carga el grafo base y la matriz O-D del día simulado.
- Por cada hora de operación (5–23h), copia el grafo, enruta cada flujo O-D con `nx.shortest_path` (ponderado por `tiempo_minutos`) y acumula `carga_pasajeros` por arista.
- Aplica **función BPR adaptada** para convertir carga en tiempo congestionado:
$$T_c = T_b \times (1 + \alpha \cdot (V/C)^\beta), \alpha=0.15, \beta=4, \text{ tope en } 4 \times T_b$$

con `CAPACIDAD_PROMEDIO_TRAMO_HORA = 35000` (constante única para toda la red).
- **[Actualizado 2026-09-27] Generador de eventos estocásticos** (reemplaza la inyección determinista de Línea 9): por cada (hora, línea, tipo de evento) del día se sortea ocurrencia con un proceso de Poisson (tasa base modulada por hora pico y, para lluvia, por temporada de lluvias), y si ocurre, severidad (Beta(2,5), sesgada a eventos leves) y duración (1-4 horas). `lluvia` afecta toda una línea de superficie (A/B/12); `falla_mecanica` e `incidente_plataforma` afectan un tramo puntual de cualquier línea. El efecto se sigue expresando como carga fantasma sobre la fórmula BPR (no se escribe en `carga_pasajeros_red` , que queda como ridership real limpio), y además se registra explícitamente en el dataset vía `hay_evento` / `tipo_evento` / `severidad_evento` .
- El ruteo ya no recalcula `nx.shortest_path` por cada fila de la matriz O-D y por cada hora: como el peso de ruteo (`tiempo_minutos`) es estático, la ruta de cada par origen-destino se calcula una sola vez y se cachea (necesario para que escalar a 14 días fuera viable en tiempo razonable: ~5.5 min totales en vez de ~14× el tiempo de un solo día).

- Construye ventanas temporales ($t-1, t, t+1$) de congestibilidad por tramo (sin fugar información entre días) y exporta `dataset_features_entrenamiento.csv`, ahora con **52,360 registros** de 14 días (antes 3,740 de un solo día).

2.4 `entrenador_anticipatorio.py` — Modelo predictivo

- Ordena cronológicamente y separa train/test 80/20 **sin aleatorizar** (evita fuga de información temporal).
- Features: `hora`, `tiempo_ideal`, `congestibilidad_t`, `congestibilidad_t_minus_1`, `hay_evento`, `severidad_evento` y dummies one-hot de `tipo_evento` (los modelos de sklearn no aceptan texto). **[2026-09-27, Sección 5]** Ubicación en la red: dummies de `linea` (derivada del ID GTFS del andén; las aristas entre líneas distintas son `transbordo`) y `tramo_congestion_media`, un target encoding suavizado del tramo (congestión futura media del tramo, calculada **solo con entrenamiento**). Target: `target_congestibilidad_t_plus_1`.
- **[2026-09-27]** Compara 6 configuraciones (`RandomForestRegressor`, `GradientBoostingRegressor`, `HistGradientBoostingRegressor`, cada una con y sin ubicación), con dos evaluaciones: el split cronológico 80/20 y una validación *rolling origin* sobre los últimos 5 días. Reporta MAE/RMSE global y, por separado, en filas con `hay_evento=1`. Elige la configuración con menor RMSE en filas con evento en la validación por días (hoy: `GradientBoosting + línea/tramo`).
- Explicabilidad por **importancia por permutación** (aumento del RMSE al barajar cada variable; las dummies de una misma variable se barajan juntas), agnóstica al modelo, en `importancia_variables.png`.
- Exporta a `modelos/modelo_anticipatorio.pkl` un paquete con el modelo, su codificación (categorías y target encoding) y el inicio del set de prueba; `ruteo_anticipatorio.py` reutiliza `construir_features()` del entrenador para no desincronizarse.

2.5 `ruteo_anticipatorio.py` — Integración estimación + ruteo

- Carga grafo base + modelo entrenado + dataset de contexto (usado como proxy de "sensores en tiempo real").
- `proyectar_hora(fecha, hora) + grafo_con_retraso(...)`: para cada arista predice el retraso de la hora siguiente y lo suma al tiempo ideal, generando un grafo proyectado. El contexto se filtra por fecha además de hora porque el dataset contiene 14 días, y las features se construyen con la misma función (`construir_features`) y codificación con que se entrenó el modelo.
- **[2026-09-27]** Los tiempos "reales" se miden contra la congestión simulada de la hora siguiente (`target_congestibilidad_t_plus_1`). Antes se medían contra la propia proyección de la IA, lo que favorecía a la IA por construcción.
- Compara ruta estática (Dijkstra sobre tiempo ideal) vs. ruta anticipatoria (Dijkstra sobre pesos proyectados por IA), evaluando ambas contra el tráfico real proyectado.
- Caso de prueba demostrado: Pantitlán → Auditorio, 7:00 AM (día laboral 2026-01-13). Con el modelo multi-día, ambos sistemas eligen la misma ruta (Línea 9 hasta Tacubaya, transbordo a Línea 7): tiempo ideal 37.94 min, tiempo real en $t+1$ 41.22 min (con el modelo `RandomForest` anterior la proyección era 40.33 min). Ya no se reproduce el desvío que mostraba la versión anterior, porque la falla fija de Línea 9 (7-9am) que lo provocaba fue reemplazada por eventos estocásticos, y ese día a esa hora no hubo un evento que congestionara esta ruta.

- **[2026-09-27]** Evaluación sistemática sobre todas las horas con evento del set de prueba (ver Sección 5.3).

3. Limitaciones metodológicas actuales (a atender antes de tesis final)

1. ~~Un solo día de simulación~~ **RESUELTO 2026-09-27** (ver [Sección 4](#)): ahora se simulan 14 días (10 laborales + 4 de fin de semana), y el split 80/20 cronológico separa *días* completos (12 de entrenamiento, 3 de prueba, con 1 día compartido en la frontera del corte), no horas dentro del mismo día.
2. ~~Evento de disrupción hardcodeado~~ **RESUELTO 2026-09-27**: la falla fija de Línea 9 fue reemplazada por un generador de eventos estocásticos (lluvia, falla mecánica, incidente de plataforma) con tasas tipo Poisson por hora/línea/temporada y severidad/duración muestreadas. Queda pendiente calibrar esas tasas contra fuentes reales (ver Sección 4.4).
3. **Capacidad constante para toda la red** (35000 pasajeros/hora): no diferencia tramos troncales de alta capacidad (Línea 1, 2, 3) de tramos periféricos.
4. **Horizonte de predicción discretizado a 1 hora**, mientras el objetivo de la tesis pide horizonte de 10–60 min. Falta granularidad sub-horaria.
5. **Perfiles origen/destino hardcodeados** (listas fijas de 10 estaciones): no se derivan de un análisis estadístico de la matriz de afluencia real, sino de un supuesto manual razonado.
6. **[2026-09-27] El modelo casi no usa las variables de evento** (Sección 5.1): la predicción depende sobre todo de la inercia (`congestibilidad_t` , `t-1`). Con la capacidad constante de 35,000 pas/h y BPR ($\beta=4$), un evento típico mueve poco la congestión, así que el efecto del evento en el target es débil.
7. **[2026-09-27] Lluvia independiente por línea**: el generador sorteaba la lluvia de las líneas A, B y 12 por separado, cuando en la realidad una tormenta afecta a varias a la vez. Además, con las tasas actuales, enero (temporada seca) tiene ~1.7 eventos de lluvia por día (Sección 5.4).

Ninguno de estos puntos invalida el trabajo — son exactamente el tipo de simplificación esperable en una primera iteración de prototipo — pero deben documentarse como alcance y quedar como candidatos directos para la siguiente fase.

4. Extensión multi-día + eventos estocásticos (2026-09-27)

Esta sección documenta la ejecución del prompt de la sesión anterior (ver historial de commits). Cambios de código:

- `generador_sintetico_horario.py` : parametrizado por `DIAS_SIMULACION` (14 fechas de enero 2026), perfil horario laboral/fin de semana por `tipo_dia` , una matriz O-D por día y un `manifiesto_dias_simulados.csv` de salida. También se restringió el histórico fuente a solo los días simulados (antes desagregaba las ~1,857 fechas del CSV completo a nivel hora sin usarlas después: un archivo intermedio de ~300 MB para un solo día útil).

- `simulador_congestion.py` : lee el manifiesto y simula los 14 días en un solo run; reemplaza la falla fija de Línea 9 por el generador de eventos estocásticos (Poisson por hora/línea/tipo, severidad Beta(2,5), duración 1-4h); cachea rutas estáticas por par origen-destino (el cuello de botella real al escalar a varios días); agrega `tipo_dia`, `hay_evento`, `tipo_evento`, `severidad_evento` al dataset.
- `entrenador_anticipatorio.py` : si el dataset trae columnas de evento, las agrega como features (`hay_evento`, `severidad_evento`, dummies de `tipo_evento`).
- `ruteo_anticipatorio.py` : **corrección detectada al verificar el pipeline completo**. El script seguía construyendo `X_pred` con las 4 features originales, mientras el modelo reentrenado espera 10, y fallaba con `ValueError: The feature names should match those that were passed during fit`. Además tenía un error silencioso: filtraba el contexto solo por `hora`, así que con 14 días cada arista quedaba con el valor del último día leído, mezclando días distintos. Ahora usa `modelo.feature_names_in_` y filtra por `fecha_viaje + hora_viaje` (caso de prueba: 2026-01-13, 7:00).

4.1 Dataset resultante

	Versión anterior (1 día, evento determinista)	Versión actual (14 días, eventos estocásticos)
Registros	3,740	52,360
Días distintos	1	14 (10 laborales, 4 fin de semana)
Filas con evento activo	~ desconocido (no etiquetado)	870 (1.66%): 707 lluvia, 94 incidente de plataforma, 69 falla mecánica
Split train/test (80/20 cronológico)	Mismo día en train y test (fuga temporal de facto)	12 días train / 3 días test (1 día compartido en la frontera del corte)
Tiempo de generación (<code>simulador_congestion.py</code>)	segundos	~5.5 min (14 días, con caché de rutas)

4.2 Cambio en MAE / RMSE

Métrica	Baseline (1 día, sin eventos aleatorios)	Nuevo (14 días, eventos estocásticos)	Δ
MAE (min)	0.0000109	0.0028	×256
RMSE (min)	0.000233	0.0313	×134

Lectura correcta de este resultado: el error subió, y eso es lo esperado y lo correcto, no una regresión del modelo. El baseline de 1 día tenía una fuga temporal de facto — el split 80/20 separaba horas del mismo día con el mismo patrón de falla recurrente (L9, 7-9am, todos los días idéntico), así que el modelo memorizaba trivialmente el patrón y el error caía a prácticamente cero ($\text{MAE} \approx 10^{-5}$ min, una precisión sin sentido físico). Con 14 días y eventos verdaderamente estocásticos (distintos tipos, tramos, horas, severidades y duraciones cada día), el modelo debe generalizar de 12 días vistos a 3 días no vistos con eventos nuevos que nunca ocurrieron exactamente igual en entrenamiento. Un MAE de ~0.003 min y RMSE de ~0.031 min sobre un target cuyo rango va de 0 a 6.09 min (media 0.015, con el 75% de los tramos en 0 — la red solo se congestiona en tramos/horas puntuales) es un desempeño

razonable para un primer modelo con features todavía limitadas (no incluye, por ejemplo, la línea/tramo como variable categórica). Esto también resuelve la limitación #1 de la Sección 3: el split ahora separa días reales, no horas del mismo día.

Importancia de variables (nuevo modelo): `congestibilidad_t` (50.3%) y `hora` (41.9%) siguen dominando; el bloque de variables de evento (`severidad_evento`, `hay_evento`, dummies de `tipo_evento`) aporta en conjunto ~1.9% — señal débil pero presente, consistente con que solo 1.66% de las filas tienen evento activo. Ampliar esa señal (más días, o sobremuestrear horas con evento) es candidato directo para la siguiente iteración si se quiere que el modelo dependa más del contexto de eventos y menos de la inercia (`congestibilidad_t`).

4.3 Supuestos de simulación explícitos (para citar en el capítulo de metodología)

- Ventana de 14 días naturales de enero 2026 (única franja con datos reales completos en el CSV fuente); no hay quincena ni variación mensual/estacional real en la muestra, solo la distinción laboral/fin de semana.
- Perfil horario de fin de semana derivado programáticamente del laboral (aplanado + desplazamiento a mediodía/tarde), no calibrado con datos reales de fin de semana.
- Tasas de eventos (Poisson), factores de hora pico/temporada de lluvias y factores de impacto por tipo de evento son **ilustrativos**, no una calibración estadística sobre incidencia real del STC (ver Sección "Análisis de factibilidad técnica" más abajo, que ya anticipaba este punto y sigue vigente para la siguiente iteración).

4.4 Siguiente paso (ejecutado — ver [Sección 5](#))

Prompt listo para usar en la siguiente sesión de trabajo:

"El dataset de entrenamiento ya tiene 14 días con eventos estocásticos, pero la señal de evento (`hay_evento` / `tipo_evento` / `severidad_evento`) solo aporta ~1.9% de importancia porque apenas 1.66% de las filas tienen un evento activo. Antes de calibrar tasas contra fuentes reales, sube la proporción de señal explotable: (1) añade `linea / tramo` como variable categórica (dummies o target encoding) para que el modelo pueda aprender qué tramos son estructuralmente más propensos a congestión, ya que hoy el modelo no sabe en qué parte de la red está parado; (2) evalúa si migrar de `RandomForestRegressor` a `GradientBoostingRegressor` (o `HistGradientBoosting`, que maneja mejor el desbalance de clases raras) mejora la predicción específicamente en las filas con `hay_evento=1` (repórtalo aparte del MAE/RMSE global, ya que hoy ese segmento es <2% de los datos y puede quedar diluido); (3) documenta en `avances.md` una propuesta concreta de fuentes para calibrar `TASA_BASE_LLUVIA`, `TASA_BASE_FALLA_MECANICA` y `TASA_BASE_INCIDENTE_PLATAFORMA` en `simulador_congestion.py` contra datos reales (climatología SMN/Conagua para lluvia; boletines de contingencia o notas de prensa del Metro para fallas/incidentes), aunque sea como aproximación documentada y no como calibración estadística rigurosa; (4) en `ruteo_anticipatorio.py`, en lugar de un solo caso fijo (Pantitlán → Auditorio, 2026-01-13 7:00, donde hoy la ruta IA coincide con la estática), recorre las fechas/horas del conjunto de prueba con `hay_evento=1`, compara ruta estática vs. anticipatoria para un conjunto de pares O-D que crucen los tramos afectados y reporta en cuántos casos la IA cambia la ruta y cuántos minutos ahorra en promedio."

5. Ubicación en la red, comparación de modelos y evaluación sistemática del ruteo (2026-09-27)

Ejecución del prompt de la Sección 4.4. Cambios de código:

- `entrenador_anticipatorio.py` : agrega la ubicación en la red como features (`línea` one-hot + `tramo_congestion_media` , target encoding suavizado calculado solo con entrenamiento); compara RandomForest / GradientBoosting / HistGradientBoosting con y sin ubicación; agrega validación *rolling origin* por días; reporta métricas en filas con evento aparte; cambia la explicabilidad a importancia por permutación; exporta un paquete (`modelos/modelo_anticipatorio.pkl` , reemplaza a `modelo_anticipatorio_rf.pkl`) con modelo + codificación + inicio del set de prueba.
- `ruteo_anticipatorio.py` : reutiliza `construir_features()` del entrenador; mide los tiempos contra la congestión real de $t+1$; agrega la evaluación sistemática sobre horas con evento (Sección 5.3), que guarda el detalle en `datos_procesados/evaluacion_ruteo_eventos.csv` .

5.1 Comparación de modelos

Métricas en minutos de retraso sobre el tiempo ideal. "Evento" = filas con `hay_evento=1` .

Split cronológico 80/20 (prueba: 16–18 de enero; 10,472 filas, **137 con evento**):

Configuración	MAE	RMSE	MAE evento	RMSE evento
HistGradientBoosting (sin ubicación)	0.0030	0.0297	0.0301	0.1236
GradientBoosting (sin ubicación)	0.0032	0.0303	0.0291	0.1263
GradientBoosting + línea/tramo	0.0039	0.0306	0.0309	0.1296
HistGradientBoosting + línea/tramo	0.0036	0.0307	0.0324	0.1346
RandomForest + línea/tramo	0.0027	0.0313	0.0364	0.1496
RandomForest (sin ubicación) — <i>modelo anterior</i>	0.0028	0.0313	0.0363	0.1541

Validación por días (5 pliegues: cada día del 14 al 18 de enero se predice con un modelo entrenado con todos los días previos; **255 filas con evento**):

Configuración	MAE	RMSE	MAE evento	RMSE evento
GradientBoosting + línea/tramo	0.0042	0.0272	0.0369	0.1193
RandomForest + línea/tramo	0.0032	0.0285	0.0488	0.1459
HistGradientBoosting + línea/tramo	0.0044	0.0311	0.0562	0.1728
HistGradientBoosting (sin ubicación)	0.0044	0.0315	0.0591	0.1801
RandomForest (sin ubicación) — <i>modelo anterior</i>	0.0036	0.0321	0.0579	0.1844
GradientBoosting (sin ubicación)	0.0043	0.0385	0.0529	0.1965
<i>Referencia: persistencia (predecir $t+1 = t$)</i>	0.0151	0.0806	0.0491	0.1802

Lectura:

1. **Las dos evaluaciones no coinciden, y la validación por días es la más confiable.** En el split 80/20 la ubicación parece no ayudar, pero ese set tiene solo 3 días y 137 filas con evento. En la validación por días (5 días, 255 filas con evento) la ubicación mejora a los tres modelos, y más en el segmento con evento: RandomForest baja su RMSE con evento de 0.184 a 0.146 (−21%) y GradientBoosting de 0.197 a 0.119 (−39%). Por eso el criterio de selección es el RMSE con evento en la validación por días.
2. **Modelo elegido: GradientBoosting + línea/tramo.** Frente al modelo anterior (RandomForest sin ubicación), en la validación por días reduce el RMSE con evento 35% (0.184 → 0.119) y el RMSE global 15% (0.032 → 0.027). A cambio, su MAE global es algo peor (0.0042 vs. 0.0036): predice retrasos pequeños mayores a 0 en tramos que no se congestionan (75% del target es 0). Para el ruteo pesa más acertar los picos de retraso, que es lo que mide el RMSE.
3. **HistGradientBoosting no mejoró sobre GradientBoosting** a pesar de lo planteado en la Sección 4.4. El problema no es tanto el desbalance como la poca señal (punto 5).
4. **Todos los modelos superan a la persistencia** en RMSE global y con evento, así que el modelo aprende algo más que "mañana igual que hoy".
5. **La señal de evento sigue siendo débil.** En la importancia por permutación del modelo elegido (aumento del RMSE al barajar la variable, set de prueba 80/20), dominan `congestibilidad_t` (0.022 min) y `congestibilidad_t_minus_1` (0.017), seguidas de `hora` (0.003) y `tramo_congestion_media` (0.003). `tipo_evento` aporta 0.0005 y `hay_evento / severidad_evento` ≈ 0 . La causa probable está en el simulador, no en el modelo: con capacidad constante y BPR, un evento típico cambia poco el retraso, y ese efecto ya queda reflejado en `congestibilidad_t` (el evento ya estaba activo en t). Se agrega como limitación #6 en la Sección 3.

5.2 Métricas del modelo exportado

El modelo exportado es `GradientBoosting + línea/tramo`, entrenado con el 80% inicial (no con todo el dataset, para que el ruteo se evalúe en días que el modelo no vio). En el 20% final: MAE 0.0039 min, RMSE 0.0306 min; en las 137 filas con evento: MAE 0.0309 min, RMSE 0.1296 min.

5.3 Evaluación sistemática del ruteo en horas con evento

Diseño. Se recorren las **27 horas con al menos un tramo en evento** del set de prueba (16, 17 y 18 de enero). Para cada hora se toman todos los pares origen-destino (una estación por nombre, 163 estaciones) cuya **ruta estática cruza un tramo con evento: 84,650 casos**. Para cada caso se calculan cuatro rutas y todas se miden contra el **tráfico real simulado de la hora siguiente**:

- **Estático:** tiempo ideal, sin información de tráfico (baseline).
- **Reactivo:** congestión observada en la hora actual t (sistema reactivo del índice 5.5).
- **Anticipatorio:** congestión que la IA proyecta para t+1.
- **Oráculo:** congestión real de t+1. Es la cota superior: el mayor ahorro posible.

Sistema	Cambia ruta	Casos que ganan / pierden vs. estático	Ahorro medio si cambia	Ahorro neto total	% del ahorro posible capturado
Reactivo	542 (0.64%)	220 / 320	-0.02 min	-8.8 min	-5.0%
Anticipatorio (IA)	282 (0.33%)	148 / 134	+0.40 min	+114.0 min	64.6%
Oráculo	234 (0.28%)	234 / 0	+0.75 min	+176.3 min	100%

Lectura:

- 1. Resultado central: el ruteo anticipatorio captura 64.6% del ahorro máximo posible; el reactivo empeora al usuario (-5%).** El sistema reactivo esquiva la congestión de la hora actual, que en la hora siguiente ya cambió (los eventos duran 1–4 h y la demanda cambia cada hora). Por eso cambia de ruta más veces (542) y pierde en más casos de los que gana (320 vs. 220). La IA cambia menos (282) y gana más de lo que pierde: +1.12 min en promedio cuando acierta y -0.39 min cuando falla. Es el argumento cuantitativo para la tesis: anticipar es mejor que reaccionar.
- 2. La magnitud absoluta es pequeña.** Solo 0.28% de los casos tiene una ruta mejor que la estática (oráculo), el ahorro máximo en un caso es 1.97 min y el viaje medio dura ~40 min. Esto viene del simulador, no del ruteo: con la capacidad constante de 35,000 pas/h y BPR, un evento casi nunca vuelve una ruta alternativa (con transbordos de 5 min) más rápida que la directa. Para defender el ahorro en tiempo, antes hay que diferenciar capacidades por tramo (limitación #3) o dar a los incidentes un efecto de cierre de tramo (ya previsto en el análisis de factibilidad, tabla 3.1).
- 3. La IA se equivoca en casi la mitad de sus cambios de ruta** (134 de 282). Esto es consistente con su RMSE con evento (~0.12–0.13 min): el ahorro disponible por caso es del mismo orden que el error del modelo.
- 4. Cautela estadística:** los 84,650 casos no son independientes. Salen de 27 horas y de pocos eventos, y muchos pares O-D comparten los mismos tramos afectados. La conclusión (anticipatorio > estático > reactivo) es consistente en esta muestra, pero para citarla con intervalos de confianza hay que repetir la simulación con varias semillas aleatorias.

5.4 Propuesta de fuentes para calibrar las tasas de eventos

Parámetros en `simulador_congestion.py`: `TASA_BASE_LLUVIA = 0.02`, `TASA_BASE_FALLA_MECANICA = 0.01` y `TASA_BASE_INCIDENTE_PLATAFORMA = 0.006` (eventos esperados por hora y por línea en hora valle), `FACTOR_HORA_PICO = 2.5` (6 horas pico) y `FACTOR_TEMPORADA_LLUVIAS = 6.0` (mayo–octubre). La operación simulada va de las 5 a las 23 h (19 horas: 13 valle + 6 pico), así que el número esperado de eventos por línea y por día es $\lambda_{base} \times (13 + 2.5 \times 6) = \lambda_{base} \times 28$.

Chequeo de orden de magnitud con los valores actuales:

Tipo	Eventos esperados/día (tasas actuales)	Referencia real
Falla mecánica + incidente de plataforma (12 líneas)	$(0.01 + 0.006) \times 28 \times 12 \approx 5.4$	3,708 incidentes con desalojo de trenes entre 2018 y agosto de 2022 , según respuesta del STC vía la Plataforma Nacional de Transparencia (El Universal): $\approx 1,704$ días $\rightarrow \approx 2.2$ por día en toda la red. La falla más frecuente fue el sistema de puertas (34 de cada 100).
Lluvia en enero (3 líneas de superficie)	$0.02 \times 28 \times 3 \approx 1.7$	Enero es temporada seca en la CDMX; los días con lluvia del mes se obtienen de las Normales Climatológicas del SMN (observatorio de Tacubaya).
Lluvia en mayo–octubre	$0.02 \times 6 \times 28 \approx 3.4$ por línea	Idem, meses de temporada de lluvias.

Las tasas de falla/incidente están en el orden correcto (unas 2.5 veces la referencia). Parte de la diferencia es esperable, porque los desalojos son solo los incidentes más graves y la referencia subestima el total. La lluvia de enero parece sobreestimada.

Fuentes propuestas y método de calibración:

- Lluvia: frecuencia diaria.** Normales Climatológicas del SMN/Conagua (<https://smn.conagua.gob.mx/es/climatologia/informacion-climatologica/normales-climatologicas-por-estado>), Ciudad de México, estación Tacubaya (observatorio de referencia del SMN): número medio de días con precipitación por mes. Con eso, $P(\text{lluvia en el día} \mid \text{mes})$ sustituye al factor binario de temporada (`FACTOR_TEMPORADA_LLUVIAS`) por un valor para cada mes.
- Lluvia: distribución horaria y extensión espacial.** Red pluviométrica del Observatorio Hidrológico del Instituto de Ingeniería UNAM (OH-IIUNAM), con unas 55 estaciones en la CDMX que registran la intensidad minuto a minuto (<https://www.iingen.unam.mx/es-mx/AlmacenDigital/Notas/Paginas/observatoriohidrologico.aspx>). Con las estaciones cercanas a las líneas A, B y 12 se estima (a) la probabilidad de lluvia por hora del día, que en la CDMX se concentra en la tarde-noche y no sigue la hora pico del Metro (hoy se le aplica `FACTOR_HORA_PICO` sin justificación física), y (b) con qué frecuencia llueve a la vez en varias líneas, para dejar de sortear cada línea por separado (limitación #7). Hay que pedir los datos al OH-IIUNAM; su publicación en tiempo real es por redes y web.
- Fallas mecánicas e incidentes de plataforma.**
 - Punto de partida documentado: la cifra de 3,708 desalojos 2018–2022 citada arriba (El Universal, <https://www.eluniversal.com.mx/metropoli/de-2018-2022-se-reportaron-3-mil-708-incidentes>), que da $\lambda_{\text{base}} \approx 2.2 / (12 \text{ líneas} \times 28) \approx 0.0065$ por línea-hora valle para ambos tipos juntos, suponiendo el mismo factor de hora pico.
 - Para separarlo **por línea, tipo y hora**: solicitud de información al STC por la Plataforma Nacional de Transparencia o su Unidad de Transparencia (<https://www.metro.cdmx.gob.mx/transparencias/transparencia-cdmx>), pidiendo el registro de incidentes/averías con fecha, hora, línea, estación y causa. Hay antecedentes de que el STC entrega este tipo de datos: la cifra anterior y el reporte de 136 averías en la Línea 12 tras su reapertura (El Gráfico, 2024).
 - Complemento (y validación del factor de hora pico): recopilación manual de los avisos de la cuenta oficial del Metro en redes sociales durante algunas semanas.

4. **Método de conversión a tasa de Poisson** (para cualquiera de las fuentes): si en D días se observaron N eventos de un tipo en una línea, con F = factor de hora pico, entonces $\lambda_{\text{base}} = N / (D \times (13 + 6 \cdot F))$. Si la fuente trae la hora del evento, F también se estima: la tasa por hora en hora pico dividida entre la tasa por hora en valle.

Esto es una **aproximación documentada, no una calibración estadística rigurosa**: los desalojos no son todos los incidentes, la cifra agrega todas las líneas y todo el periodo 2018–2022, y la climatología de Tacubaya representa un solo punto de la ciudad. Aun así, cada parámetro queda respaldado por una fuente citable en lugar de un valor ilustrativo.

5.5 Siguiente paso

Prompt listo para usar en la siguiente sesión de trabajo:

"La evaluación sistemática (Sección 5.3) muestra que el ruteo anticipatorio captura 64.6% del ahorro posible y el reactivo empeora (–5%), pero el ahorro absoluto es pequeño (el oráculo solo mejora 0.28% de los casos, máximo 1.97 min) porque, con capacidad constante y BPR, los eventos casi no cambian la ruta óptima; además el modelo apenas usa las variables de evento. Ataca la causa en el simulador: (1) sustituye `CAPACIDAD_PROMEDIO_TRAMO_HORA` por una capacidad por línea (trenes por hora $\times$ capacidad del tren, con valores documentados del STC: frecuencia de paso y tipo de tren por línea); (2) haz que `incidente_plataforma` pueda cerrar el tramo temporalmente (peso muy alto o arista removida en `G_hora`) en lugar de solo agregar carga fantasma; (3) aplica las tasas de la Sección 5.4 ($\lambda_{\text{base}} \approx 0.0065$ combinado para falla+incidente, lluvia por mes según días con precipitación de Tacubaya, sin factor de hora pico para lluvia, y lluvia correlacionada entre las líneas A, B y 12); (4) regenera el dataset, reentrena (el entrenador ya compara modelos) y vuelve a correr `ruteo_anticipatorio.py`. Repite la simulación con al menos 5 semillas para reportar la media y el intervalo del % de ahorro capturado por cada sistema, y actualiza `avances.md`."

Análisis de factibilidad técnica: eventos estocásticos (lluvia, contingencias/accidentes)

Nota (2026-09-27): el análisis de esta sección ya fue implementado — ver [Sección 4](#). Se conserva completo porque documenta el razonamiento y las fuentes consideradas para calibrar las tasas de eventos, que sigue siendo trabajo pendiente (Sección 4.4).

Conclusión general: es técnicamente factible y compatible con la arquitectura existente, con esfuerzo moderado. El diseño actual ya separa la "capa de infraestructura" (grafo estático), la "capa de demanda" (matriz O-D) y la "capa de estrés" (función BPR + inyección de carga), lo que da puntos de extensión naturales para insertar estocasticidad sin rediseñar el sistema desde cero.

3.1 Dónde se integra cada tipo de evento

Evento	Efecto físico a modelar	Punto de integración en el código actual
Lluvia	Reduce velocidad de acceso a estaciones de superficie/elevadas (Línea A, B, 12 en tramos), incrementa afluencia por trasvase modal (gente que deja camión/bici por metro)	Multiplificador probabilístico sobre <code>peso_atractividad / pasajeros_viaje</code> en <code>generador_sintetico_horario.py</code> , y/o reducción de <code>CAPACIDAD_PROMEDIO_TRAMO_HORA</code> en tramos de superficie dentro de <code>simulador_congestion.py</code>
Falla mecánica / contingencia de línea	Reduce o anula capacidad de un tramo específico por tiempo variable	Ya existe el mecanismo (inyección de carga fantasma en <code>simulador_congestion.py</code>); solo falta parametrizarlo estocásticamente en vez de hardcodearlo
Accidente/incidente de plataforma	Cierre temporal de estación/tramo (grafo con disponibilidad reducida, no solo más lento)	Requiere nueva función que remueva temporalmente una arista o nodo del grafo (<code>G_hora.remove_edge(...)</code>) en vez de solo penalizar el peso — el grafo actual no maneja indisponibilidad total, solo congestión

3.2 Requisitos técnicos adicionales

- Generador de eventos por proceso estocástico.** Un proceso de Poisson (o Binomial Negativa si se quiere sobredispersión) por hora/línea es suficiente y barato computacionalmente: para cada hora simulada, se sortea si ocurre un evento y, si ocurre, se sortea tipo, tramo afectado, severidad y duración. Esto es perfectamente compatible con el bucle `for hora in horas_operacion` que ya existe en `simulador_congestion.py`.
- Fuente de probabilidades realistas.** Aquí está el reto principal, no el técnico sino el de datos: no hay acceso (según la justificación de la propia tesis) a un feed en tiempo real de incidentes del STC Metro. Opciones viables:
 - Usar reportes históricos públicos (redes sociales oficiales del Metro, prensa, boletines de contingencia) para estimar frecuencia aproximada de incidentes por línea — trabajo de recolección manual/scraping, factible en el tiempo de tesis a escala pequeña (decenas de eventos, no miles).
 - Alternativamente (y es lo recomendable dado el alcance de "prototipo validado con datos sintéticos" que ya declara el anexo), **parametrizar la probabilidad a mano con justificación teórica/documental** (p. ej. "la Línea A reporta X incidentes por trimestre según nota de prensa Y"), dejando explícito que es una aproximación y no una calibración estadística rigurosa. Esto es consistente con el marco metodológico ya aceptado (datos sintéticos, no datos reales en tiempo real).
 - Para lluvia específicamente, sí existe una fuente pública razonable: climatología histórica de la CDMX (SMN/Conagua) para estimar la probabilidad de lluvia fuerte por mes/hora, que es más defendible que inventar una tasa arbitraria.
- Grafo con disponibilidad variable.** El código actual (`grafo_base_metro.gexf`, copiado cada hora) soporta perfectamente eliminar/restaurar aristas por snapshot, ya que cada `G_hora = G_base.copy()` es independiente. No se requiere cambio estructural de librería (sigue siendo NetworkX), solo lógica adicional.

4. **Impacto en el modelo predictivo.** Añadir eventos estocásticos exige ampliar las features del `RandomForestRegressor` (o migrar a un modelo que maneje mejor variables categóricas/exógenas, ej. Gradient Boosting con variables dummy de evento). El riesgo real es de **datos, no de modelo**: con solo 3,740 filas de un día, meter eventos raros (baja frecuencia por diseño) puede dejar clases extremadamente desbalanceadas. Esto refuerza la necesidad del paso previo (varios días simulados) antes de sumar estocasticidad, para tener suficientes muestras positivas de "hubo evento" en el set de entrenamiento.
5. **Costo computacional.** Trivial. Simular N días $\times$ 19 horas $\times$ un sorteo de evento por hora es una operación de milisegundos; el cuello de botella sigue siendo el ruteo de la matriz O-D con `nx.shortest_path` por cada par origen-destino, que ya corre hoy sin problema para un día.

3.3 Riesgos y mitigaciones

- **Riesgo:** sobre-ingeniería del generador de eventos distrayendo del objetivo central (ruteo anticipatorio). *Mitigación:* limitar el generador a 2-3 tipos de evento (lluvia, falla de tramo, cierre de estación) con una distribución simple, documentando explícitamente que el objetivo es probar que el sistema **reacciona correctamente ante disponibilidad variable del grafo**, no producir un simulador meteorológico/operativo exhaustivo.
- **Riesgo:** que la probabilidad "inventada" reste rigor metodológico ante el jurado. *Mitigación:* citar fuente para cada parámetro (aunque sea aproximado) y encuadrarlo explícitamente como supuesto de simulación, tal como ya se hace con los perfiles origen/destino y la función BPR actual — el anexo mismo justifica el uso de simulación por la dificultad de acceso a datos reales en tiempo real.

Veredicto: la integración es factible dentro del alcance de la tesis y no requiere nueva infraestructura ni librerías; el trabajo pendiente es principalmente de diseño de parámetros (tasas de ocurrencia) y de expandir el dataset a múltiples días para que el modelo tenga suficientes ejemplos de eventos disruptivos que aprender.

Mensaje de commit

feat: comparar modelos con ubicacion en red y evaluar ruteo en horas con evento

- `entrenador_anticipatorio.py`: agrega linea (one-hot) y target encoding de tramo (solo con train); compara RandomForest, GradientBoosting y HistGradientBoosting con y sin ubicacion, en split 80/20 y validacion por dias (5 pliegues); reporta metricas aparte en filas con evento; elige por RMSE con evento (GradientBoosting + linea/tramo: 0.184 -> 0.119 vs. modelo anterior); importancia por permutacion; exporta paquete `modelos/modelo_anticipatorio.pkl` (reemplaza `modelo_anticipatorio_rf.pkl`).
- `ruteo_anticipatorio.py`: reutiliza `construir_features` del entrenador; mide rutas contra la congestion real de $t+1$ (antes contra la proyeccion de la IA); evalua estatico/reactivo/anticipatorio/oraculo en 27 horas con evento de dias no vistos (84,650 casos O-D): la IA captura 64.6% del ahorro posible, el reactivo -5%.
- `avances.md`: seccion 5 con resultados y propuesta de fuentes para calibrar tasas de eventos (SMN Tacubaya, OH-IIUNAM, desalojos STC via PNT).